

Simulated backend for frontend

Testing without simulated users or expensive setups

AI-assisted tests with confidence, without relying on real backend setups.

Disclaimer

There is no silver bullet. This is **one** way, the one that worked for us.

Approaches we consider	PROS	CONS
Full backend locally	Real business rules and responses. Catches integration bugs early	Must run databases, queues, caches and other services. Grows with each new backend dependency until it's tricky, or impossible, to run fully
Shared dev environment	Zero local setup. Closest to production	Shared, unstable data that changes under your feet. May be down when you need it. Needs real users and tokens, plus a way to roll back what tests create
Testcontainers & co.	Real services, reproducible and disposable per run	Heavy images and slow startup, especially on CI. You still need to know how to run and seed every backend service
Simulated backend	Very fast. Fully isolated, no credentials, no shared state	Only as true as the contract we agreed on. Real backend bugs go unnoticed, and the frontend team has to maintain it

We wanted an approach that could be run in local and in CI environments without requiring any fake user account or tokens, the simulated backend was what worked best for us.

The trade-off we accept

- We test against **what we agreed** with the backend, not against the real backend
- If the backend changes without telling us, or is down... our tests won't notice but they will still pass as do not require a backend
- **And that's fine.** We want:
 - Fast tests, locally and on CI
 - Run the test on before push
 - A well-tested frontend that can catch issues and verify behavior against the agreed contract

Requirements

- **An API contract** (in our case, OpenAPI). This is the only hard requirement
- **Read access to the backend source**, so AI can learn the business rules the contract doesn't describe
- Our stack (pick your own):
 - Vitest + Vitest Browser Mode, real browser, real DOM
 - MSW intercepts requests at the network level
 - @msw/data, an in-memory database for the mocks
 - An AI agent using the **skill** we developed to help us create/maintain a simulated backend

How it started

"Let's just mock the endpoints with MSW, how hard can it be?" 😊

Mocking every endpoint

Plain MSW knows nothing about our API:

```
1 http.get("/api/v1/collections/:id", () => HttpResponse.json({ nmae: "oops" })); // ✅ compiles
```

- What exactly should each endpoint return?
- When the contract changes, how do we know which mocks are now wrong?

So we built a small wrapper on top of MSW's `http`, typed from the OpenAPI schema:

```
1 import type { paths } from "./schema"; // generated from OpenAPI
2
3 export function http<P extends keyof paths, M extends Methods<P>>(<
4   path: P, // only paths that exist in the contract
5   method: M, // only methods that path supports
6   resolver: HttpResponseResolver<
7     PathParams<P, M>,
8     RequestBody<P, M>,
9     ResponseBody<P, M>
10   >,
11   ): HttpHandler;
```

Regenerate the schema → **TypeScript** tells us which mocks broke 🎉

Now for real, mock every single endpoint

But wait... to run the app locally we also need to **get past authentication** 🗝️

Lucky for us, our component library (Amplify) already supports a **mock user**. We only had to plug MSW in before the app renders:

```
1  async function enableApiMocking() {
2    if (import.meta.env.VITE_IS MOCK !== "true") return;
3    const { worker } = await import("../api/msw/browser");
4    return worker.start({ onUnhandledRequest: "bypass" });
5  }
6
7  void (async () => {
8    await enableApiMocking();
9    createRoot(document.querySelector("#root")).render(<App />);
10 })();
```

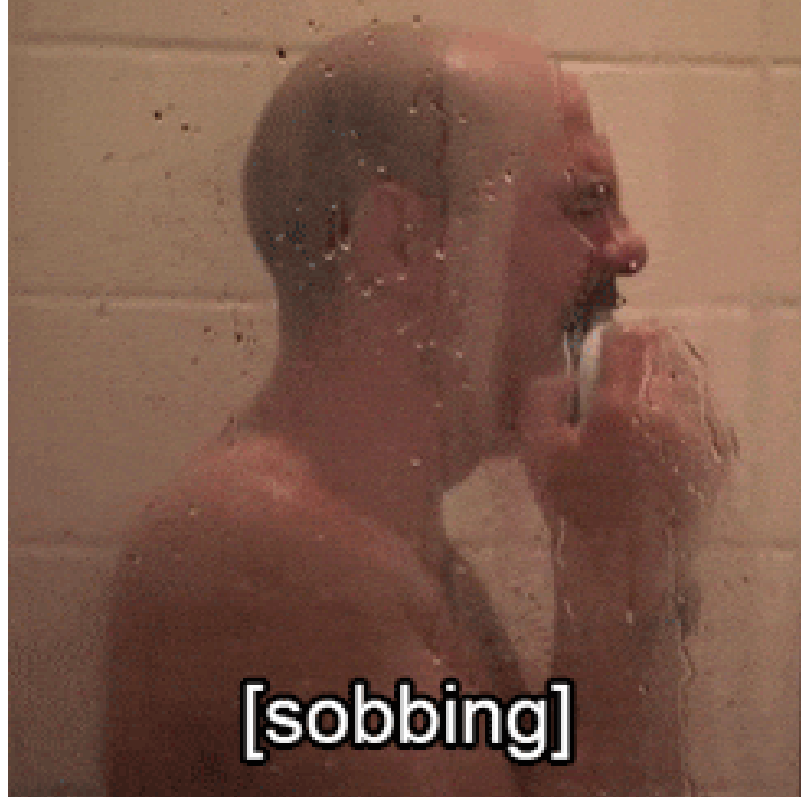
VITE_IS MOCK=true → no login, no tokens, no real backend. Same app, same code 🎉

Now for real real!! mock every single endpoint

```
1 export const getCollectionById = http(  
2   "/api/v1/collections/{collectionId}",  
3   "get",  
4   async ({ params }) => {  
5     const collection = inMemoryCollections.get(params.collectionId);  
6     if (!collection) return HttpResponse.json(null, { status: 404 });  
7     return HttpResponse.json(collection);  
8   },  
9 );
```

- Looks easy...
- ...until you have **dozens of endpoints** to mock
- ...and the backend **keeps changing them** 😬

Congratulations. You own two backends now 🎉



Then the tests grew...

- More features → more tests → more mocks. We're now at **~1500 tests**
- We were constantly overriding handlers inside tests:

```
1  worker.use(  
2    http("/api/v1/fields/{fieldId}/collections-overview", "get", () =>  
3      HttpResponse.json([createMockCollection({ field: mockFields[0] })]),  
4  ),  
5  );
```

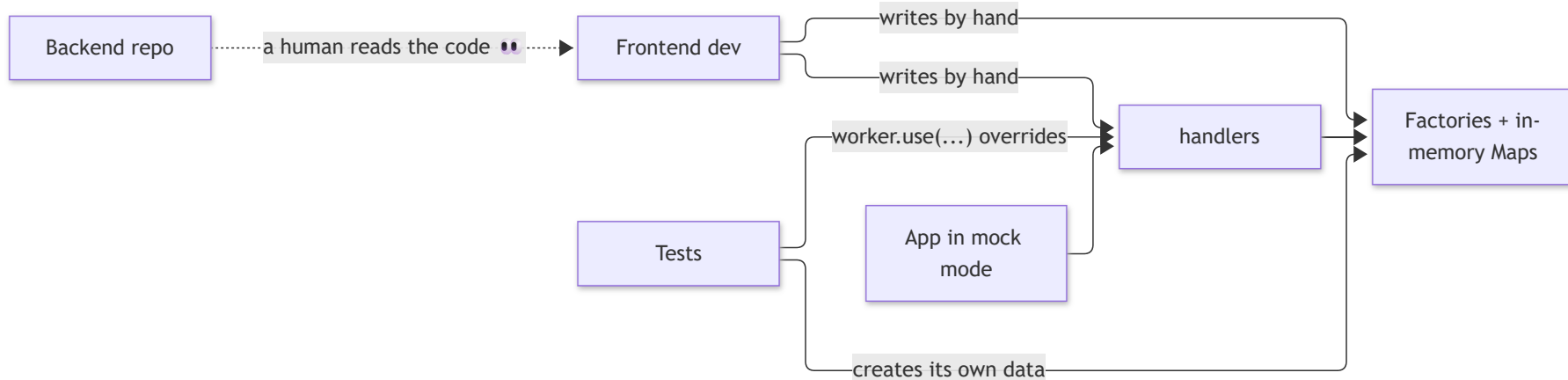
- We started writing tests **with AI...** and AI loved to mock **everything** 🤖
- Tests passed and checked what we asked for, but they tested the mocks, not the app
- We had to review every line: does this test even make sense?
- **Seeding data was painful.** Every factory was handwritten and data lived in plain JS `Map`
- Want a screen with all its data? Fill every map yourself and hope the relations match 🙏

A very green test but what does it actually verify?

```
1  const createCollection = vi.fn().mockResolvedValue({
2    collectionId: "1",
3    title: "My collection",
4  });
5
6  await createCollection({ title: "My collection" });
7
8  expect(createCollection).toHaveBeenCalledWith({ title: "My collection" });
```

No form. No API client. No app. The test checks its own mock 🙄

What we had



- One human is the sync mechanism between backend and mocks
- Tests, dev mode and data all touch the same pieces

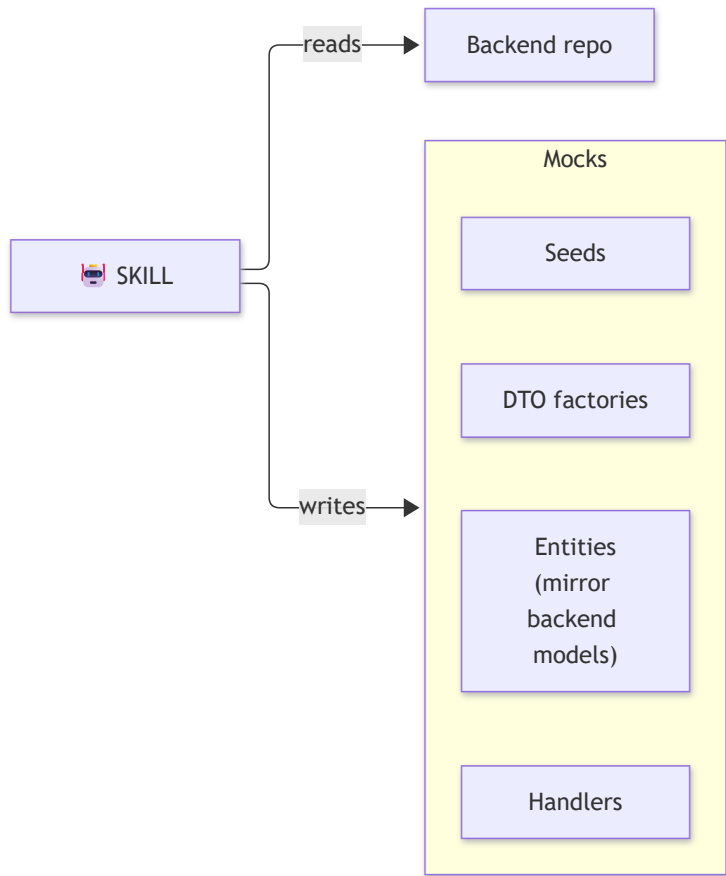


What if AI maintains it?

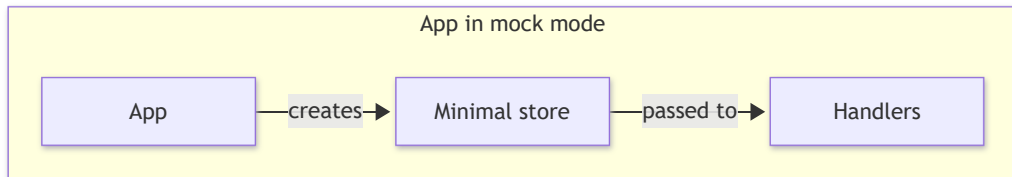
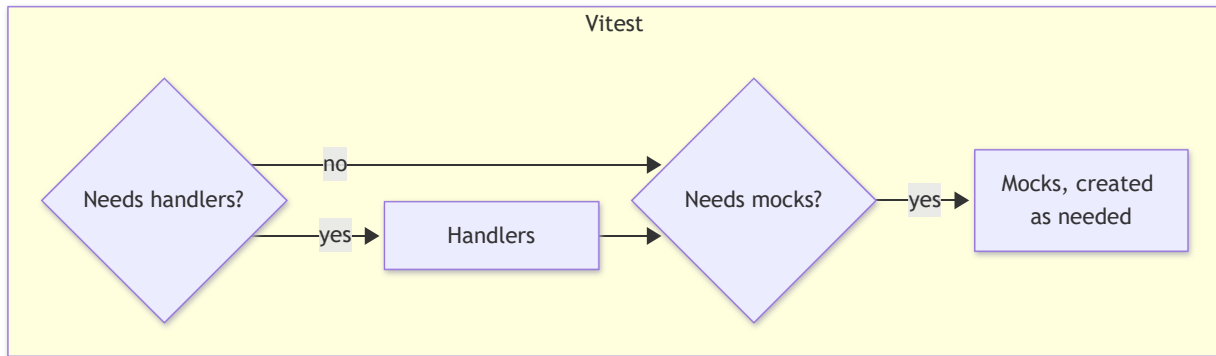
AI is very good at reading code, **far faster than any of us**

So... let it read the backend and keep the mocks in sync?

The idea: AI writes/maintains the simulated backend



The idea: who uses it



Showing the Skill

Disclaimer, still a work in progress.

But... It worked! 🎉

- The skill kept handlers and mocks in sync with the backend
- Then we thought: **what if AI drives the whole app** to build and validate a task?
- We don't want to give AI a token, a user, or access to **any** real system
- We already had the pieces: mock mode, a mock user, MSW. They just weren't structured for it

It worked, however writing a skill is a lot of try/error and we realized AI is not good at writing skills.

Give the skill a database: @msw/data

```
1 // schemas/collection.ts
2 export const collectionSchema = z.object({
3   collectionId: z.guid(),
4   title: z.string(),
5   field: fieldSchema,
6   createdBy: userSchema,
7   // ...
8 }) satisfies z.ZodType<CollectionDto>;
```

```
1 // index.ts
2 export function createMockDatabase() {
3   const users = new Collection({ schema: userSchema })
4   const collections = new Collection({ schema: collectionSchema })
5   collections.defineRelations(({ one, many }) => ({
6     field: one(fields),
7     createdBy: one(users),
8     datasets: many(datasets),
9   }));
10  return { users, fields, collections, datasets };
11 }
```

- Schemas typed against the OpenAPI DTOs (`satisfies`)
- Real relations, queries and writes, like a database

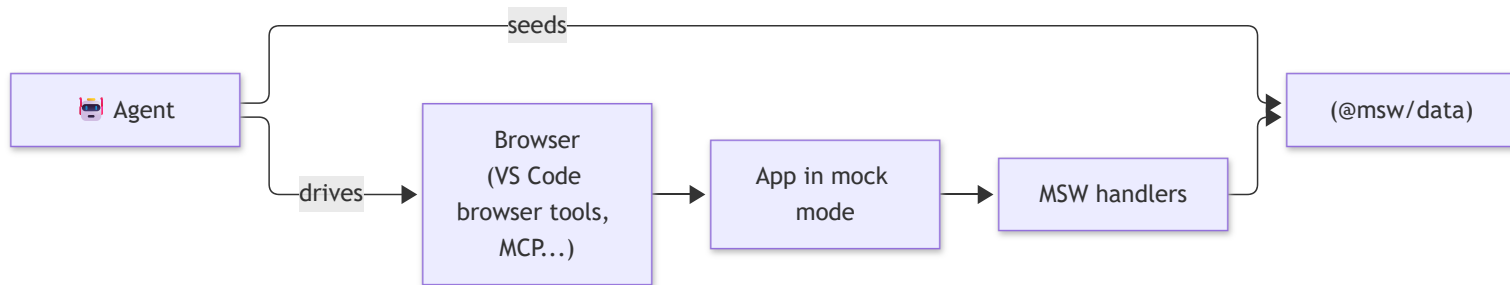
Seeds

```
1  export async function seedMockDatabase(db: MockDatabase) {
2    faker.seed(123); // same data every run
3    const creator = await db.users.create(createUser());
4    const approver = await db.users.create(createUser());
5    const field = await db.fields.create(
6      createField({ assetApprovers: [approver] }),
7    );
8    const dataset = await db.datasets.create(createDataset({ approver }));
9    await db.collections.create(
10     createCollection({ field, createdBy: creator, datasets: [dataset] }),
11   );
12 }
```

- **Mock mode:** create a database, seed it, run the app
- **Tests:** a fresh database per test, seeded with only what the test needs
- Seeding and testing are finally separate

Where we are now

AI can start the application without any real backend or real user data and seed all the data it needs to develop and test any feature or fix a bug.



- **No tokens, no users, no access** to any real system
- Data is disposable: break it, reset it, seed again
- AI can now **develop, test and validate** a task end to end quite easily
- Want security? Run the whole setup/code/AI in a separate VM

A real case: `DatasetDataTable`

An Excel-like table inside our app

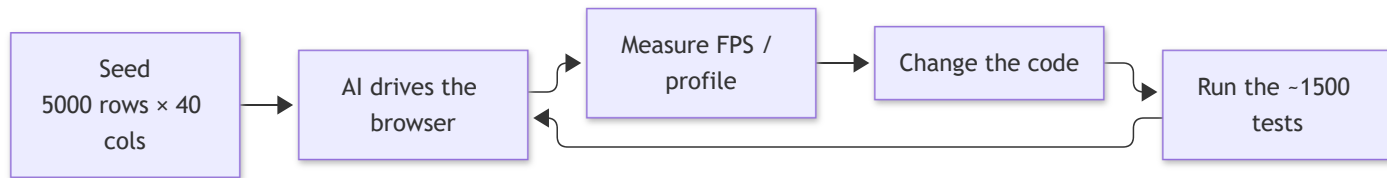
The problem

- Full keyboard navigation, range selection, copy/paste, inline editing, undo/redo
- Sorting, grouping, aggregations, hiding columns
- And it **struggled to render even a modest dataset** 🐌

"Just use virtualization!"

Yes, you must. But virtualization alone doesn't fix it when every keystroke, selection or sort can re-render the whole grid.

How the flow helped



- AI seeded the heavy dataset itself. No production data needed
- It iterated alone: measure, refactor, re-run tests, measure again
- Result: virtualization + **per-cell subscriptions**, so only the cells that change re-render.

Result

5000 rows × 40 columns = 200,000 cells at **60 fps** 🚀

But we could render even larger datasets, starting to see issues passing 1 million cells.

Demo

The app running on the simulated backend, and the code behind it

Takeaways

- An **OpenAPI contract** + backend code is all AI needs to keep a simulated backend in sync
- AI gets a full app to develop, test and validate tasks with **no access to real systems**
- With **no great power** comes **no great responsibility**
- It's a trade-off: you test against the contract, not the real backend. **Choose it consciously**

Thanks!